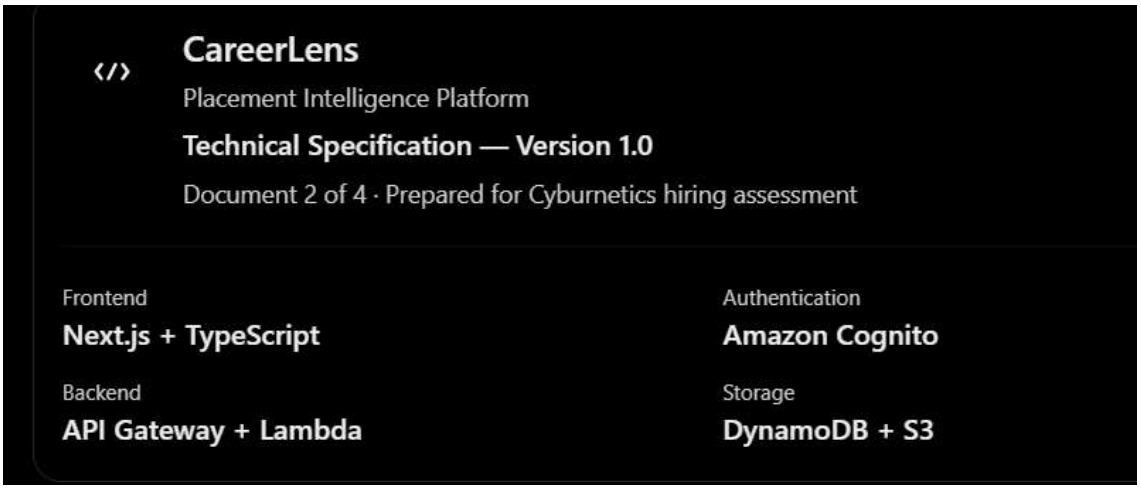


1. Purpose and scope

This document defines how CareerLens will be implemented at the frontend and backend levels. It establishes the technology choices, project structure, responsibilities of each module, authentication flow, API integration approach, data validation rules, security controls, and environment configuration.

The objective is to deliver a functional, maintainable application suitable for an intermediate-level student hiring assessment without introducing unnecessary infrastructure or advanced services.

Technical design principles

- **Use TypeScript throughout the frontend and backend where practical.**
- **Keep frontend components separate from business logic.**
- **Validate all untrusted data on the backend.**
- **Enforce authorization on every user-owned resource.**
- **Use DynamoDB for structured application data and S3 for resume files.**
- **Keep AWS credentials and secrets out of frontend code.**
- **Deploy without Amazon EC2.**

2. Technology stack

Layer	Selected technology	Responsibility
Web application	Next.js App Router	Pages, layouts, routing
Programming language	TypeScript	Type safety
Styling	Tailwind CSS	Responsive UI
UI components	Reusable React components	Consistent interface
Form validation	React Hook Form + Zod	Form state and client validation
Authentication	Amazon Cognito User Pools	Registration, login, identity
API	Amazon API Gateway HTTP API	HTTPS endpoints and JWT authorization
Business logic	AWS Lambda, Node.js + TypeScript	Server-side operations

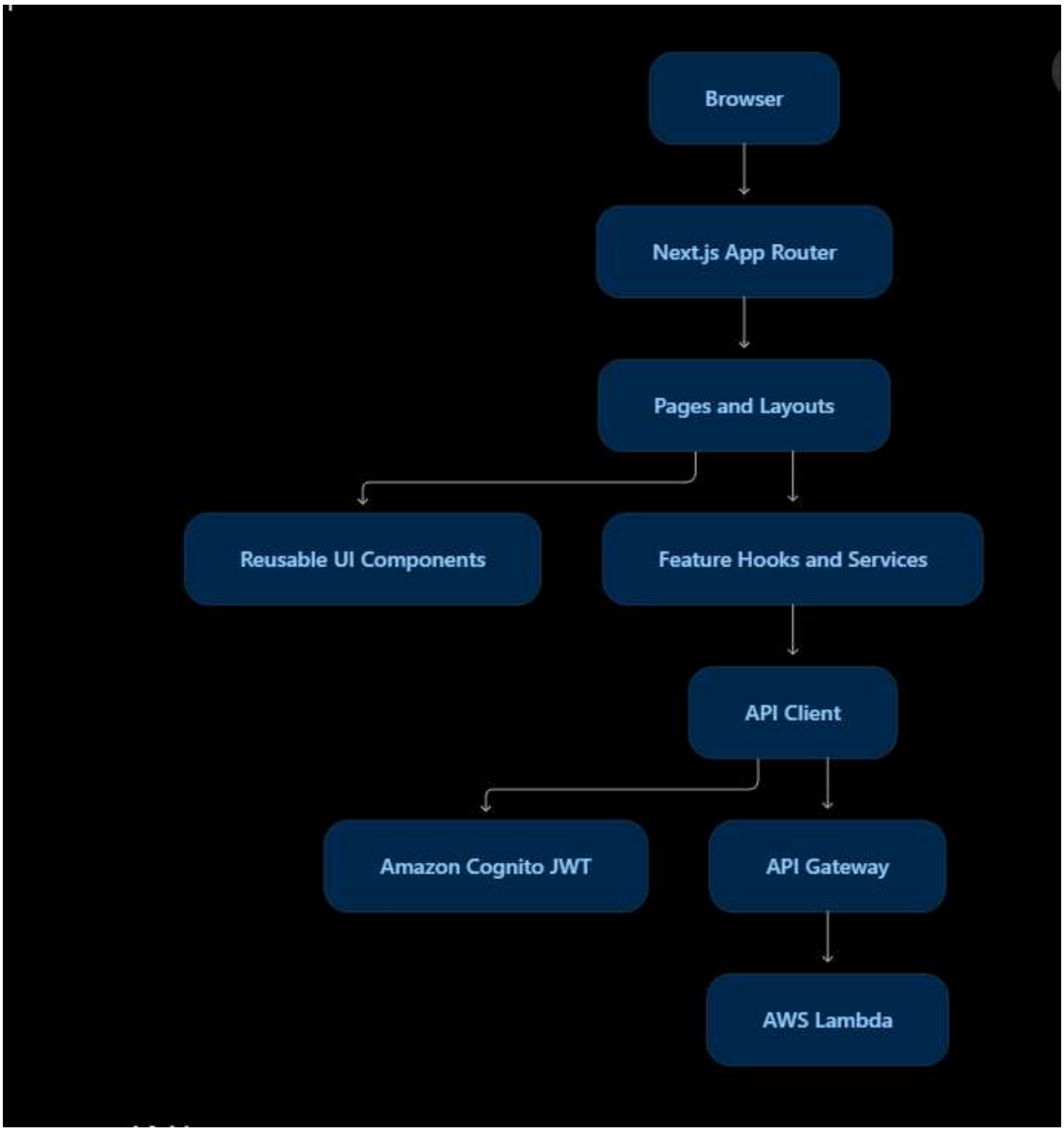
Database	**Amazon DynamoDB**	**User profiles, applications, interviews**
File storage	**Amazon S3**	**Private resumes and associated files**
Hosting	**AWS Amplify Hosting**	**Frontend deployment**
Monitoring	**Amazon CloudWatch**	**Logs and operational diagnostics**
Unit testing	**Vitest**	**Business logic and utility tests**
UI testing	**React Testing Library**	**Component behavior**
API testing	**Vitest or an HTTP client**	**Endpoint integration tests**

Implementation decision: Use Next.js with the App Router rather than React Native because the deliverable is a browser-based web application. Use Amazon Cognito for identity, API Gateway for protected API access, and Lambda for business logic.

3. Frontend technical specification

3.1 Frontend architecture

The frontend is responsible for rendering pages, collecting user input, displaying API responses, and managing temporary UI state. It must not contain AWS secret keys or directly perform privileged database operations.



3.2 Proposed folder structure

```
careerlens/
├── public/
```

```
│   ├── images/
│   └── icons/
├── src/
│   ├── app/
│   │   ├── layout.tsx
│   │   ├── page.tsx
│   │   ├── globals.css
│   │   ├── (auth)/
│   │   │   ├── login/page.tsx
│   │   │   └── register/page.tsx
│   │   └── (dashboard)/
│   │       ├── layout.tsx
│   │       ├── dashboard/page.tsx
│   │       ├── applications/page.tsx
│   │       ├── applications/new/page.tsx
│   │       ├── applications/[id]/page.tsx
│   │       ├── skill-analyzer/page.tsx
│   │       ├── interviews/page.tsx
│   │       ├── resumes/page.tsx
│   │       └── profile/page.tsx
│   ├── components/
│   │   ├── ui/
│   │   ├── layout/
│   │   ├── dashboard/
│   │   ├── applications/
│   │   ├── skills/
│   │   ├── interviews/
│   │   └── resumes/
│   ├── features/
│   │   ├── auth/
│   │   ├── applications/
│   │   ├── skills/
│   │   ├── interviews/
│   │   └── resumes/
│   ├── hooks/
```

```
│   ├── lib/
│   │   ├── api-client.ts
│   │   ├── auth.ts
│   │   ├── env.ts
│   │   └── formatters.ts
│   ├── schemas/
│   │   ├── application.schema.ts
│   │   ├── profile.schema.ts
│   │   └── interview.schema.ts
│   └── types/
│       ├── application.ts
│       ├── profile.ts
│       └── interview.ts
├── tests/
│   ├── unit/
│   └── integration/
├── .env.example
├── package.json
├── tsconfig.json
└── README.md
```

This structure is a proposed starting point. Keep feature-specific files together where possible; do not create empty folders solely to make the project appear more complex.

3.3 Page and component responsibilities

Page or component	Responsibility
Login and registration	Authenticate users through Cognito

Dashboard	**Show application statistics and upcoming interviews**
Application list	**Search, filter, sort, and paginate records**
Application form	**Create and update job application information**
Application detail	**Show job description, status history, and related interviews**
Skill analyzer	**Compare user skills with required job skills**
Interview calendar/list	**Manage interview schedules, notes, and outcomes**
Resume vault	**Upload, list, and securely download resume versions**
Profile	**Update personal details and skill inventory**
Sidebar and navigation	**Provide consistent application navigation**

Status badge	Display recruitment stage consistently
Loading and error states	Communicate API progress and failures

3.4 Frontend state management

For the MVP, use React state and feature-level hooks rather than introducing Redux or another global state library.

Use:

- Local component state: modal visibility, selected filters, form controls.
- Feature hooks: application lists, profile data, interviews, resume metadata.
- Authentication state: current signed-in user and token/session status.
- Server data: fetched from the backend, with explicit loading, success, and error states.
- URL search parameters: filters and pagination that users should be able to bookmark or share.

Avoid treating local storage as the source of truth for private application data. The backend must persist all authoritative records.

3.5 Frontend API client

All backend calls should go through a centralized API client.

Responsibilities:

1. Obtain a valid Cognito access token using the supported authentication SDK.

2. Attach the token as a bearer token to protected API requests.
3. Serialize request bodies and parse JSON responses.
4. Handle HTTP error codes consistently.
5. Abort obsolete requests where appropriate.
6. Avoid logging access tokens or sensitive profile data.

Example response contract:

```
export type ApiError = {
  code: string;
  message: string;
  requestId?: string;
};

export type ApiResponse<T> = {
  data: T;
};
```

The exact API response envelope will be finalized in Document 3. Do not wrap successful responses inconsistently across endpoints.

3.6 Frontend validation

Use Zod schemas with React Hook Form to provide immediate feedback.

Example:

```
import { z } from "zod";

export const applicationSchema = z.object({
  companyName: z.string().trim().min(1).max(120),
  jobTitle: z.string().trim().min(1).max(120),
  jobDescription: z.string().max(20000),
  status: z.enum([
    "APPLIED",
```

```
    "ASSESSMENT",
    "INTERVIEW",
    "OFFER",
    "REJECTED",
    "WITHDRAWN",
  ]),
});
```

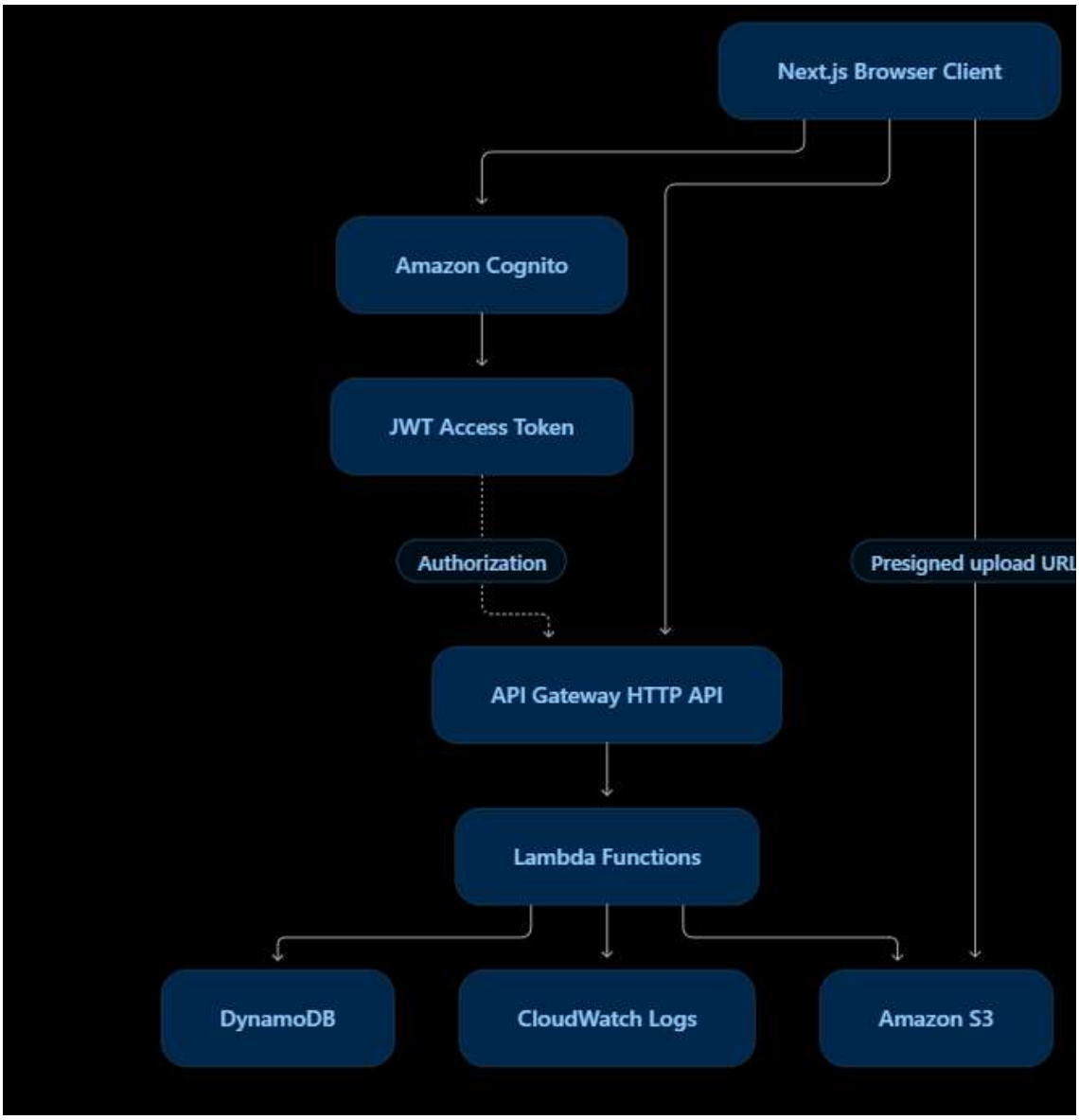
This improves the user experience but does not replace backend validation. The backend must independently enforce the same constraints and any additional business rules.

3.7 Frontend performance

- Prefer Server Components for static layouts and noninteractive content.
- Use Client Components only for interactive features.
- Paginate application lists rather than rendering every record.
- Lazy-load nonessential visualizations.
- Avoid unnecessary API requests on each keystroke; debounce search.
- Use appropriately sized images and fonts.
- Display skeletons or loading states for asynchronous operations.
- Measure production performance rather than relying only on development mode.

4. Backend technical specification

4.1 Backend architecture



The backend is responsible for authorization, business rules, data validation, database access, and controlled file operations.

4.2 Lambda function organization

For a beginner-friendly MVP, use a small number of cohesive Lambda functions rather than creating a separate function for every tiny operation.

Lambda module	Main responsibilities
`profile-handler`	Get and update the authenticated user's profile and skills
`applications-handler`	List, create, retrieve, update, and delete applications
`skill-analysis-handler`	Normalize skills, calculate matches, and identify gaps
`interviews-handler`	Create, list, update, and delete interview records
`resumes-handler`	Create presigned upload/download URLs and manage metadata
`dashboard-handler`	Return summary metrics and upcoming interview information

Use API Gateway routes to direct requests to the appropriate handler. Related operations may share a handler with explicit method and route dispatching.

Important: Every handler must derive the user identity from the verified Cognito JWT claims supplied by API Gateway, not from a `userId` submitted by the browser.

4.3 Backend request lifecycle

Each protected request should follow this sequence:

1. **API Gateway receives the HTTPS request.**
2. **The Cognito JWT authorizer validates the token.**
3. **API Gateway invokes the mapped Lambda function.**
4. **Lambda extracts the authenticated user identity from trusted authorizer claims.**
5. **The handler validates path parameters, query parameters, and request body.**
6. **The handler verifies resource ownership and applicable business rules.**
7. **Lambda reads or writes DynamoDB, or performs an authorized S3 operation.**
8. **Lambda returns a consistent HTTP response.**
9. **CloudWatch captures relevant operational logs without secrets.**

API Gateway authorization should reject invalid or expired tokens before the request reaches business logic.

5. Amazon Cognito authentication

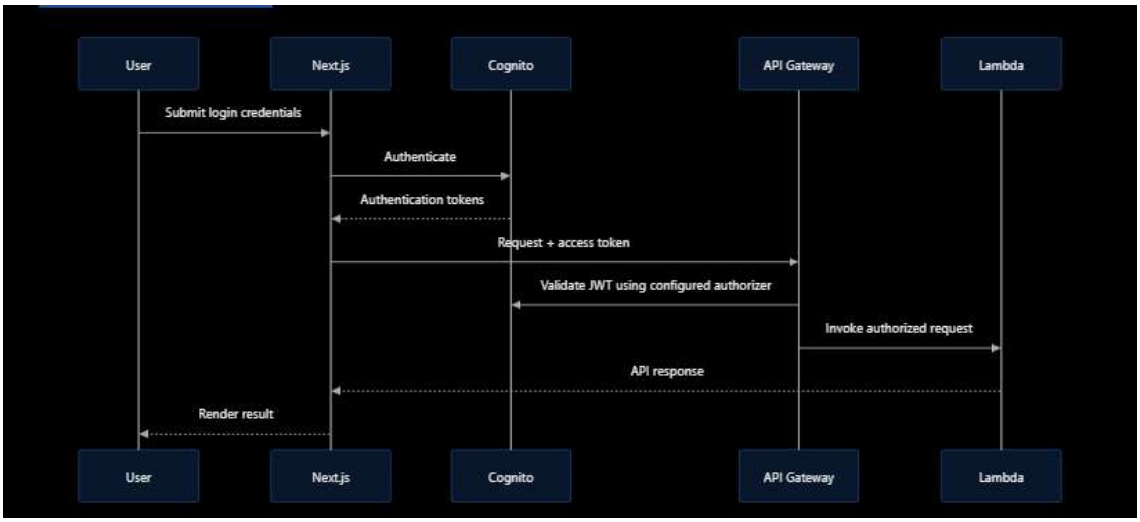
5.1 Authentication responsibilities

Use a Cognito User Pool for:

- **User registration and login.**
- **Password reset.**
- **Email verification, if enabled in the user pool.**
- **Issuing JWT tokens.**
- **Authentication session management.**

Use the supported Cognito SDK for the selected frontend authentication flow. For a browser-only application, use a secure, documented flow and follow Cognito's recommendations for token storage and session handling.

5.2 Authentication flow



5.3 Authentication versus authorization

Authentication answers: “Who is the user?”

Authorization answers: “What is this user allowed to access?”

A valid token alone does not authorize a user to access every record. Every Lambda handler must check that the requested application, interview, resume, or profile belongs to the authenticated user, unless an explicitly authorized administrative capability applies.

5.4 Session security

- **Do not expose refresh tokens in URLs or logs.**
- **Never embed AWS secret access keys in the browser.**
- **Use HTTPS for production requests.**
- **Configure appropriate session expiration and logout behavior.**
- **Keep access tokens out of application logs and analytics events.**
- **Verify Cognito token types and claims are appropriate for API authorization.**

6. DynamoDB integration

6.1 Data responsibilities

DynamoDB stores structured application data:

- **User profile and skills.**
- **Job applications and their recruitment status.**
- **Interview schedules and outcomes.**
- **Resume metadata and S3 object keys.**
- **Skill-analysis results, if persisted.**

Resume file contents themselves belong in S3, not in DynamoDB items.

6.2 Data access implementation

Use the AWS SDK for JavaScript v3 with DynamoDB Document Client in Lambda.

Recommended practices:

- **Use environment variables for table names.**
- **Define partition and sort keys deliberately.**
- **Use `Query` for known access patterns instead of broad `Scan` operations.**
- **Paginate results.**
- **Use conditional writes where concurrent updates could overwrite data.**
- **Store UTC timestamps in a consistent ISO 8601 representation.**
- **Avoid unnecessary duplicate storage of derived metrics.**

6.3 Data ownership

A record must be associated with its owner. The API should scope queries by the authenticated identity or verify ownership before accessing an item.

For example, retrieving an application by ID must not succeed simply because the ID exists. The handler must verify that the application belongs to the caller.

The exact partition keys, sort keys, indexes, and query patterns will be defined in Document 3.

7. Amazon S3 resume management

7.1 Storage design

Use a private S3 bucket for resume files. DynamoDB stores metadata, including the resume ID, owning user, object key, file size, version, and upload timestamp.

7.2 Upload workflow

1. **The user selects a PDF resume.**
2. **The frontend requests an upload URL from the backend.**
3. **Lambda verifies authentication, ownership, and allowed upload constraints.**
4. **Lambda generates a short-lived presigned S3 URL for a server-generated object key.**
5. **The browser uploads directly to S3.**
6. **The frontend confirms the upload.**
7. **The backend verifies the uploaded object and persists metadata.**

A presigned URL is a temporary capability: anyone possessing it may use it within its allowed permissions and validity period. It must not be exposed in logs or stored as permanent resume metadata.

7.3 File validation and security

For the MVP, accept PDF resumes only.

- **Enforce a maximum upload size, such as 5 MB.**

- Generate object keys server-side.
- Validate actual file content where practical; do not trust only the filename or MIME header.
- Keep the bucket private and block public access.
- Use short expiration times for presigned URLs.
- Require authenticated access to generate download URLs.
- Ensure the requested resume belongs to the current user.
- Configure bucket CORS only for the deployed frontend origin.
- Apply suitable encryption and lifecycle policies.

The application should not allow arbitrary user-supplied object keys to determine which S3 object is downloaded.

8. Validation and error handling

8.1 Validation responsibilities

Input	Validation
Company name	Required, trimmed, maximum 120 characters
Job title	Required, trimmed, maximum 120 characters
Job description	Maximum 20,000 characters
Application status	Must match an allowed enum

Interview date	Valid date/time, stored consistently
Skills	Normalized strings, reasonable item and length limits
Resume upload	PDF-only policy, size limit, ownership checks
Resource ID	Valid format and authorized ownership
Search and pagination	Bounded page size and validated query parameters

These limits are initial design choices and can be adjusted during implementation.

8.2 Standard error categories

HTTP status	Meaning	Example
`400`	Invalid request	Missing required field
`401`	Unauthenticated	Missing or invalid token

`403`	Forbidden	User lacks required permission
`404`	Resource not found	Application does not exist or is inaccessible
`409`	Conflict	Conditional write conflict
`413`	Payload too large	Upload exceeds allowed limit
`429`	Too many requests	Rate limit exceeded
`500`	Internal server error	Unexpected backend failure

Return safe, user-friendly messages. Never send raw stack traces, internal AWS error details, secrets, or database configuration to the browser.

9. Environment variables and configuration

Keep separate configurations for local development and AWS deployment.

9.1 Frontend environment example

Create `.env.local` for local development and exclude it from version control.

```
NEXT_PUBLIC_APP_NAME=CareerLens
NEXT_PUBLIC_API_BASE_URL=https://your-api-id.execute-api
.your-region.amazonaws.com
NEXT_PUBLIC_COGNITO_USER_POOL_ID=your-user-pool-id
NEXT_PUBLIC_COGNITO_CLIENT_ID=your-app-client-id
NEXT_PUBLIC_AWS_REGION=your-region
```

The values above are placeholders. Replace them with actual resources created in your AWS account.

Variables prefixed with `NEXT_PUBLIC_` are exposed to browser code. Only publish values intended to be public, such as the Cognito user pool ID, app client ID, AWS region, and API base URL.

Never put AWS secret access keys, database credentials, private signing keys, or other secrets in `NEXT_PUBLIC_` variables.

9.2 Lambda environment example

```
AWS_REGION=your-region
DYNAMODB_TABLE_NAME=careerlens-main
S3_RESUME_BUCKET=your-private-resume-bucket
APP_ENV=production
LOG_LEVEL=INFO
```

The Lambda execution role should grant the minimum necessary DynamoDB and S3 permissions. AWS supplies execution credentials through its runtime environment; do not manually place access keys in Lambda environment variables.

9.3 Configuration rules

- Commit `.env.example` with placeholder values only.
- Add local secret files to `.gitignore`.
- Configure deployment values in the relevant AWS service settings.

- **Use separate resources for development and production where feasible.**
- **Avoid logging environment variables.**
- **Restrict CORS to known frontend origins in production.**
- **Record the deployed resource names and configuration in the private deployment notes, not in public logs.**

10. Backend code organization

A lightweight Lambda project can use this structure:

```
careerlens/
├── frontend/
│   └── src/
│       ├── app/
│       ├── components/
│       ├── features/
│       └── lib/
├── backend/
│   ├── src/
│   │   ├── handlers/
│   │   │   ├── profile.ts
│   │   │   ├── applications.ts
│   │   │   ├── skills.ts
│   │   │   ├── interviews.ts
│   │   │   ├── resumes.ts
│   │   │   └── dashboard.ts
│   │   ├── services/
│   │   ├── repositories/
│   │   ├── schemas/
│   │   ├── middleware/
│   │   ├── utils/
│   │   └── types/
│   ├── tests/
│   └── package.json
├── infrastructure/
```

```
│   └── README.md
├── docs/
│   ├── PRD.md
│   ├── TECHNICAL_SPEC.md
│   └── ARCHITECTURE_API.md
├── .env.example
└── README.md
```

The frontend and backend can use separate `package.json` files to manage their dependencies independently.

Backend module responsibilities

- Handlers: Interpret API Gateway requests and create HTTP responses.
- Services: Implement application rules, skill matching, and workflow logic.
- Repositories: Encapsulate DynamoDB read and write operations.
- Schemas: Validate requests and data structures.
- Middleware/utilities: Shared parsing, error handling, and logging.
- Types: Shared backend domain models.

Do not introduce a large framework or multiple layers without a concrete need. The structure should make the code easier to test and explain.

11. Performance and scalability decisions

CareerLens is a student-focused MVP, so the architecture should optimize for simplicity and predictable performance rather than premature scaling.

1. Keep Lambda handlers small and reuse initialized SDK clients across warm invocations.

2. Use DynamoDB query patterns that match the actual application screens.
3. Limit page sizes and use continuation tokens for pagination.
4. Avoid fetching every application merely to calculate dashboard statistics.
5. Use S3 direct uploads so large file bytes do not pass through Lambda.
6. Avoid unnecessary React re-renders and repeated API calls.
7. Add caching only for suitable non-sensitive data.
8. Use CloudWatch logs to identify slow requests and errors.
9. Set reasonable Lambda timeouts and API throttling limits.
10. Verify the production deployment and test latency under realistic usage.

12. Development implementation order

1. Initialize the frontend. Create the Next.js TypeScript application, layouts, navigation, and responsive page skeletons.
2. Configure authentication. Set up Cognito and protect the dashboard routes.
3. Deploy the first API. Create API Gateway, a Lambda handler, and the DynamoDB table; verify one authenticated request end to end.
4. Implement application management. Add create, list, view, update, and delete operations with ownership enforcement.
5. Implement skill analysis. Add deterministic skill normalization, comparison, and match percentage calculation.
6. Add interviews and dashboard. Build the related records and summary views.

7. **Add resume management. Implement presigned uploads/downloads and verified metadata storage.**
8. **Test and deploy. Verify authentication, validation, access control, production hosting, and all critical workflows.**

13. Definition of done

Document 2's technical design is considered implemented when:

- **The frontend and backend have clear module boundaries.**
- **Cognito authentication is connected to protected API Gateway routes.**
- **Lambda validates requests and enforces record ownership.**
- **DynamoDB persists application data.**
- **S3 supports private resume uploads and downloads.**
- **Environment configuration contains no exposed AWS secrets.**
- **Error responses are consistent and useful.**
- **The frontend handles loading, success, and failure states.**
- **The selected AWS hosting configuration works without EC2.**
- **Critical frontend and backend workflows can be tested independently.**